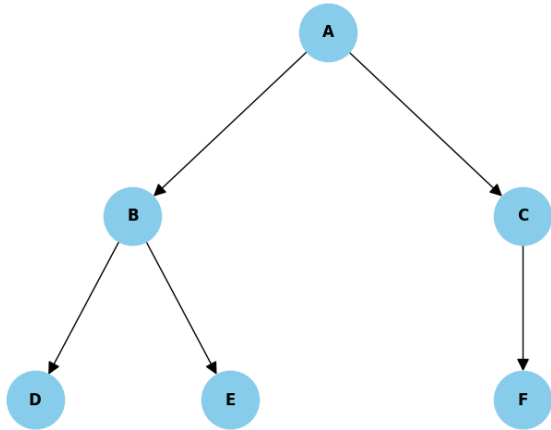


Lab 2

Write a basic program to implement Depth First Search Algorithm

Example problem



Here, A is the initial state and F is the final state

Pseudocode

BEGIN

DEFINE adj_list as:

'A' → ['B', 'C']

'B' → ['D', 'E']

'C' → ['F']

'D' → []

'E' → []

'F' → []

DEFINE status_codes:

1 → Ready (Unvisited)

2 → Waiting (In Stack)

3 → Visited (Popped from Stack)

FUNCTION initialize_status(adj_list):

FOR each node IN adj_list:

SET status[node] = 1

RETURN status

FUNCTION dfs_search(start, dest, adj_list):

```
SET stack = empty list
SET final_path = empty list
SET status = initialize_status(adj_list)
```

```
PUSH start onto stack
SET status[start] = 2
```

```
WHILE True:
    SET current = POP top of stack
    APPEND current to final_path
    SET status[current] = 3
```

```
IF current = dest THEN
    BREAK
```

```
FOR each neighbor IN adj_list[current]:
    IF status[neighbor] = 1 THEN
        PUSH neighbor onto stack
        SET status[neighbor] = 2
```

```
IF stack is empty THEN
    BREAK
```

```
RETURN final_path
```

```
FUNCTION display_adjacency_list(adj_list):
    PRINT "Adjacency List:"
    FOR each key, value IN adj_list:
        PRINT key → value
```

```
FUNCTION main:
    CALL display_adjacency_list(adj_list)
```

```
INPUT start node
INPUT destination node
```

```
CONVERT both to uppercase
```

```
CALL dfs_search(start, destination, adj_list)
STORE result in path
```

```
PRINT "Path from [start] to [destination]:"
FOR each node IN path:
```

PRINT node →
PRINT "End"

CALL main IF this is the main program

END

Report format

Title: Write a program

Date: Lab assigned date

Objective

- To implement the Depth First Search (DFS) algorithm using an adjacency list to explore nodes in a graph or tree structure.
- To find and display the path from a given start node to a destination node, if such a path exists, using DFS traversal.

Theory

- Depth First Search (DFS) Algorithm

Implementation

- Write down the problem statement
- Python Code
- Output (Screenshots of your output)

Conclusion